Back To Basics
Containers
Part 2: New and Future Containers

ALAN TALBOT



2026
September 12 - 18

Introduction

- I will be here at the end for questions
  - If we don’t get to yours, post them to the **back-to-basics-containers** discord channel
  - Or look for me in the hall!
- C++ Standard Library containers, adaptors and improvements
  - In C++17, C++23 and C++26
- Alan Talbot
  - Using C++ almost exclusively for over 35 years
  - On the C++ committee (WG21) for 21 years

Contiguous Containers

- Block-based containers with only one memory block
  - The block may be dynamically allocated on the heap
  - Or local (contained within the container object)
- The elements in the memory block are adjacent
  - So they have optimal iteration semantics
  - And optimal locality/predictability of reference
  - Yielding the best possible iteration performance
- **vector**
  - This is a dynamic sequence container (discussed in Part 1)
- **inplace_vector**
  - This is a local sequence container (up next)
- **array** is considered a container, but it’s an aggregate

inplace_vector

- Added to the Standard Library in C++26
- A local-storage contiguous sequence container
  - Like **boost::static_vector**
- The elements are embedded in the container object itself
  - Instead of dynamically allocated on the heap
- The capacity is fixed at compile time
- It can have *dramatically* better performance than vector
  - Both space and time
- It mimics the behavior of **vector** as closely as possible
  - Including omitting front operations, so it can't be used with **queue**

inplace_vector

- You can choose its reaction to an attempt to exceed the capacity
- For example, if the size is equal to the capacity:
  - **push_back** throws **std::bad_alloc**
    - With the strong guarantee
    - Useful if the failure is unlikely and exceptional
  - **try_push_back** returns an empty **std::optional**
    - The return type is **std::optional<value_type&>**
    - Useful if the failure is part of normal operation
  - **unchecked_push_back** invokes undefined behavior
    - Most likely a crash
    - Useful if the failure is impossible absent a programming bug

inplace_vector<element, 5>



Block-based Containers

- Block-based containers that can have *more* than one memory block
  - The blocks are dynamically allocated on the heap
- Emptied blocks may or may not be deallocated
  - This is implementation defined
- **deque**
  - This is a sequence container (discussed in Part 1)
- **hive**
  - This is not quite a sequence container (up next)

hive – The Bee's Knees?

- Added to the Standard Library in C++26
  - Originally called "colony"
  - A specific implementation of a more general data structure
- A block-based, semi-contiguous, unordered "sequence" container
- It offers some advantages of both block-based and node-based containers:
  - Fast iteration
  - *O*(1) insertion to an unspecified position
    - Slow if a new block is allocated
  - *O*(1) deletion anywhere
    - Slow if an empty block is deallocated
  - Reuse of space freed by deletion
  - Stability—existing elements are not moved unless requested
    - So links are not invalidated

hive

- Not quite a sequence
  - It’s in the *sequences* section of the standard, but doesn't model a sequence
  - Unordered (unless you sort it)
  - Does not support random access
- If all the blocks are full, a new block is allocated on insertion
  - Block size grows by an implementation-defined factor (e.g. 2.0)
  - But block size can have user-defined min and max limits
- As elements are deleted, their vacated storage space is retained
  - It is then reused for subsequent insertions
  - If emptied blocks are retained, no deallocation so no fragmentation

hive

- Elements are stable unless specific operations are performed
  - Insertion does not move elements
  - Deletion does not move elements
  - Some functions explicitly move elements and therefore invalidate links
    - **shrink_to_fit**, **reshape**, **sort**, etc.
- Iteration is fast but (usually) arbitrary
  - Iteration skips missing elements using a very clever algorithm
  - Iteration order is undefined except immediately following a sort
  - The order you insert doesn't guarantee the order of iteration
    - Even for fill/range insertions

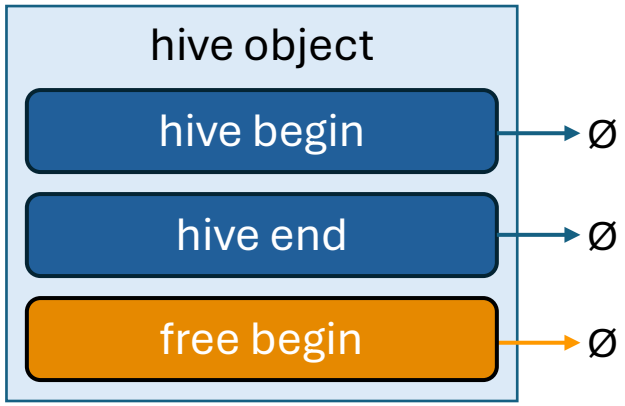
hive – Use Cases

- Ideal for use cases which:
  - Have no ordering requirement
  - Need fast iteration
  - Need efficient storage
  - Are volatile (lots of insertions and deletions)
  - Require stability (elements are linked to or can't move for some other reason)
- For Example
  - Gaming – this is the industry the hive/colony project came from originally
  - Particle simulation (weather, physics)
  - Financial data manipulation (especially high-frequency trading)
- Despite its association with gaming, **hive** is a general-purpose container
  - Similar structures have long been used by many industries

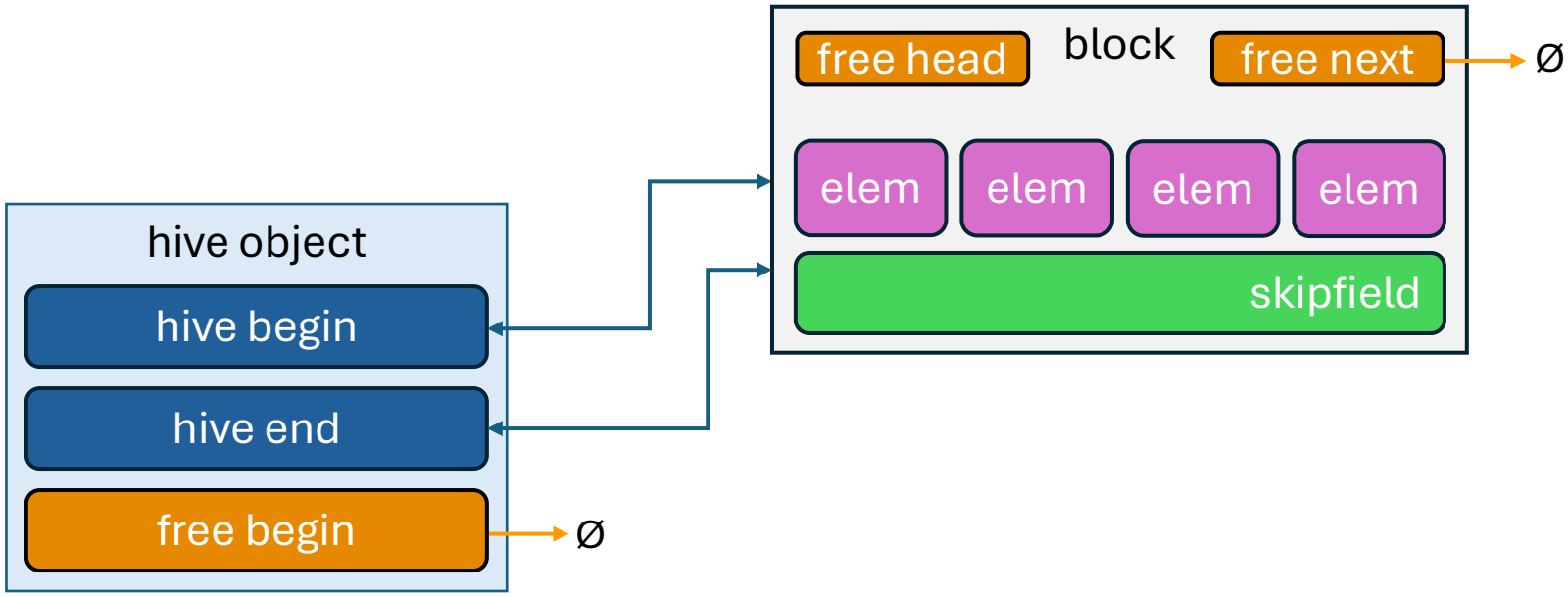
hive – Reference Implementation Details

- A double-linked list of blocks
  - Blocks hold the elements
- Each block has a skip field
  - This provides the information that allows the iterator to have *O*(1) complexity and still jump gaps formed by multiple consecutive deletions
- Each block has an internal free list
  - This lets the free list be implemented with indices rather than pointers, to save space
- There is a second double-linked list of blocks with free space
  - This, combined with the internal free lists, makes finding a free space *O*(1)

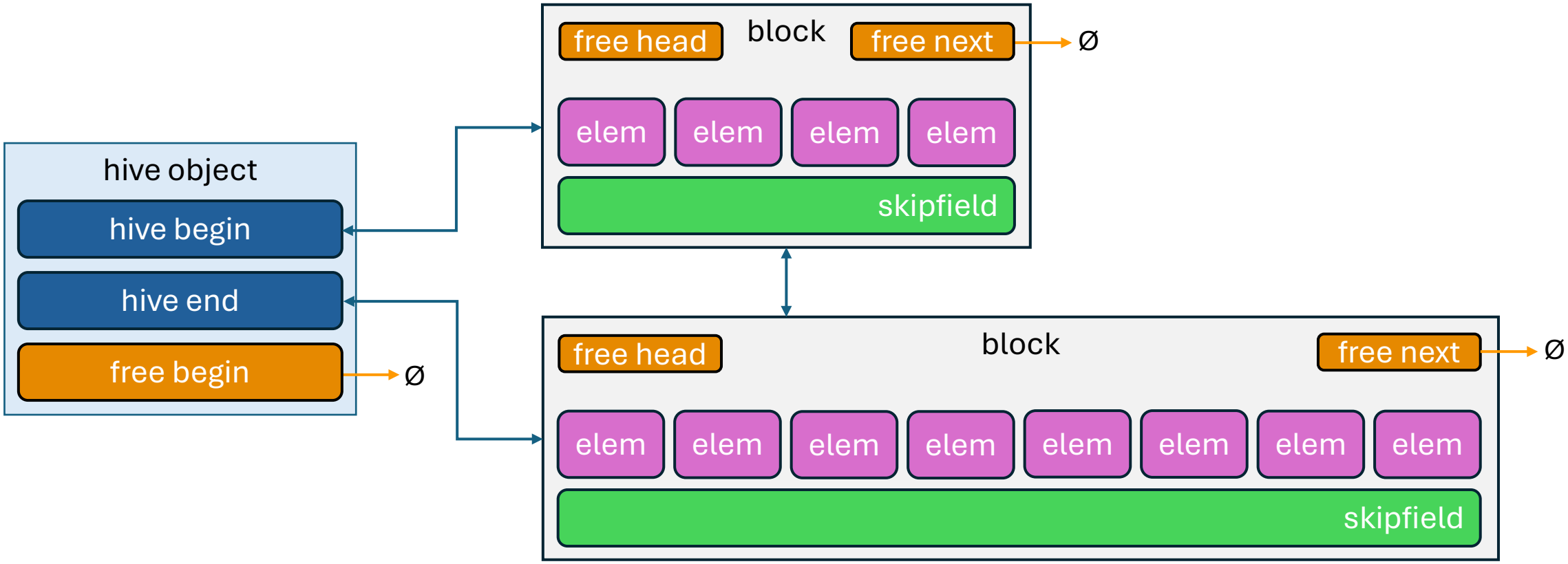
hive – Default Constructed



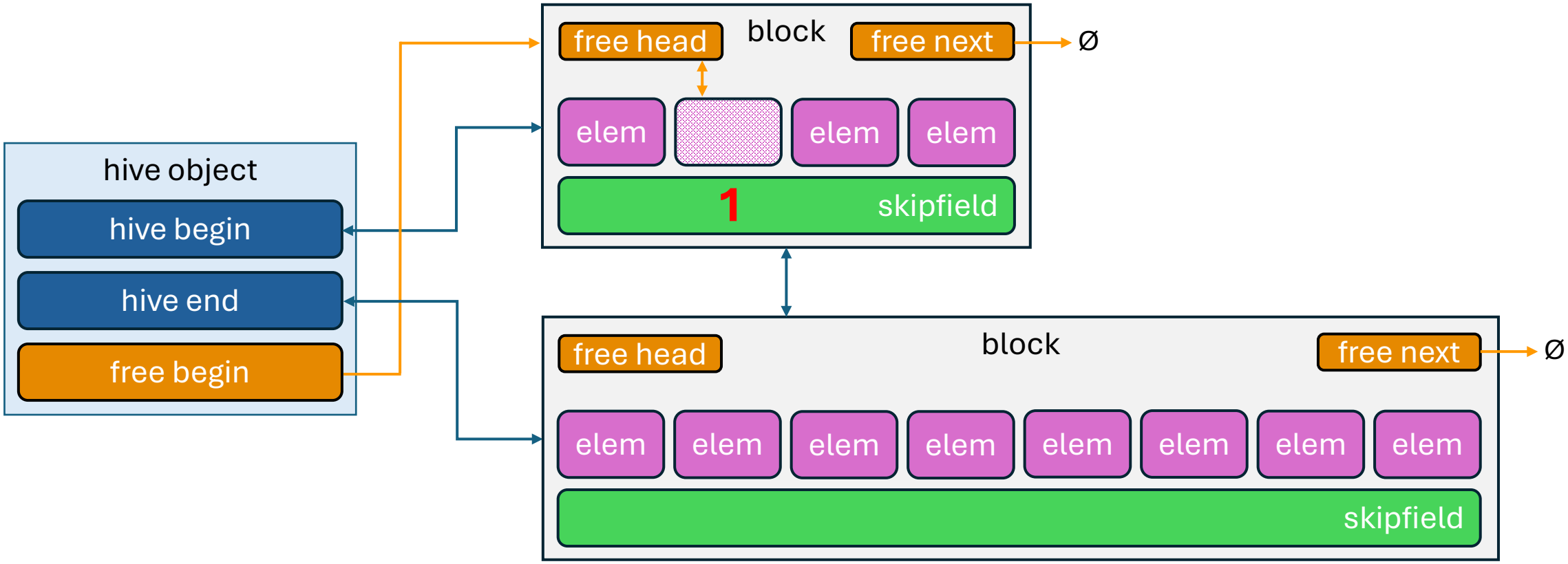
hive – First Block Created



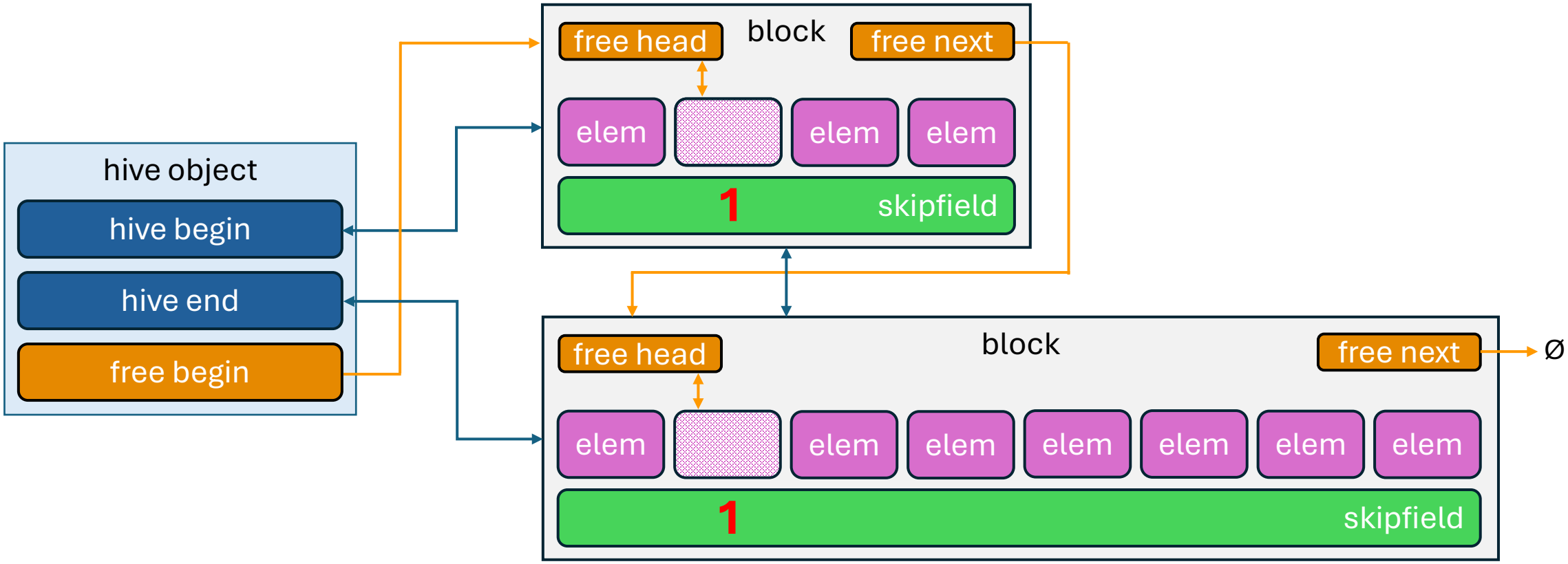
hive – New Block Needed



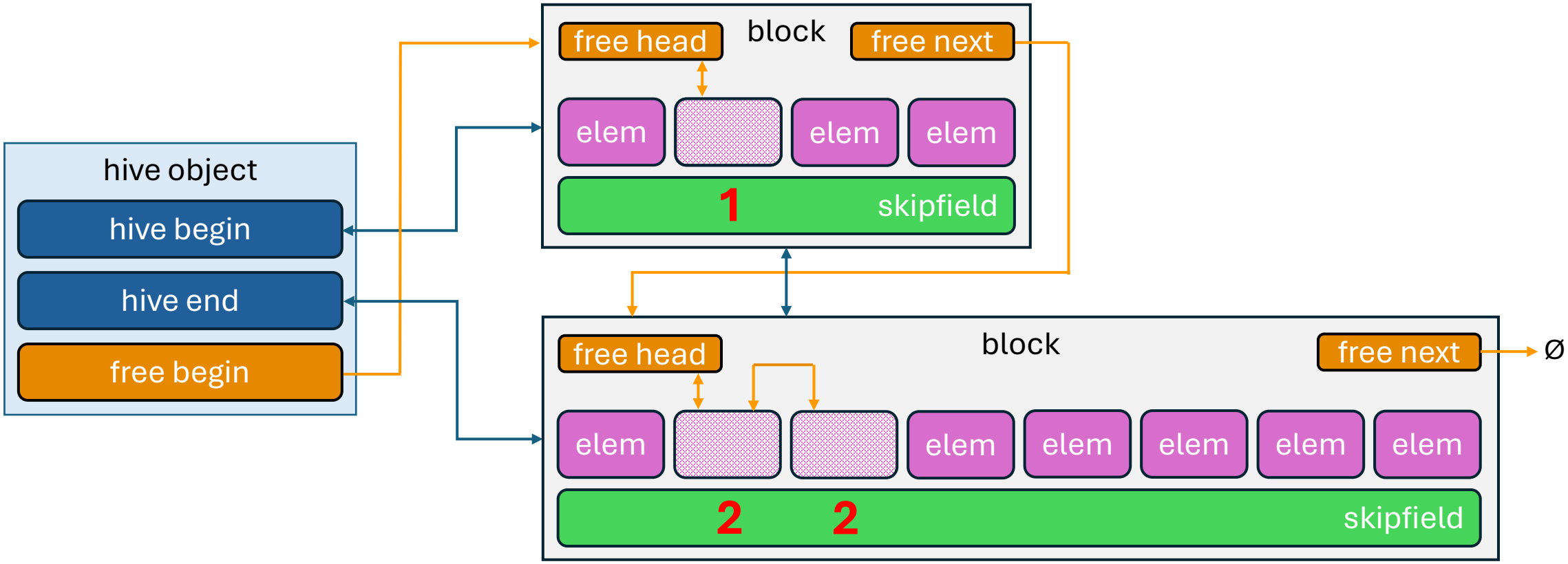
hive – Deleting Elements



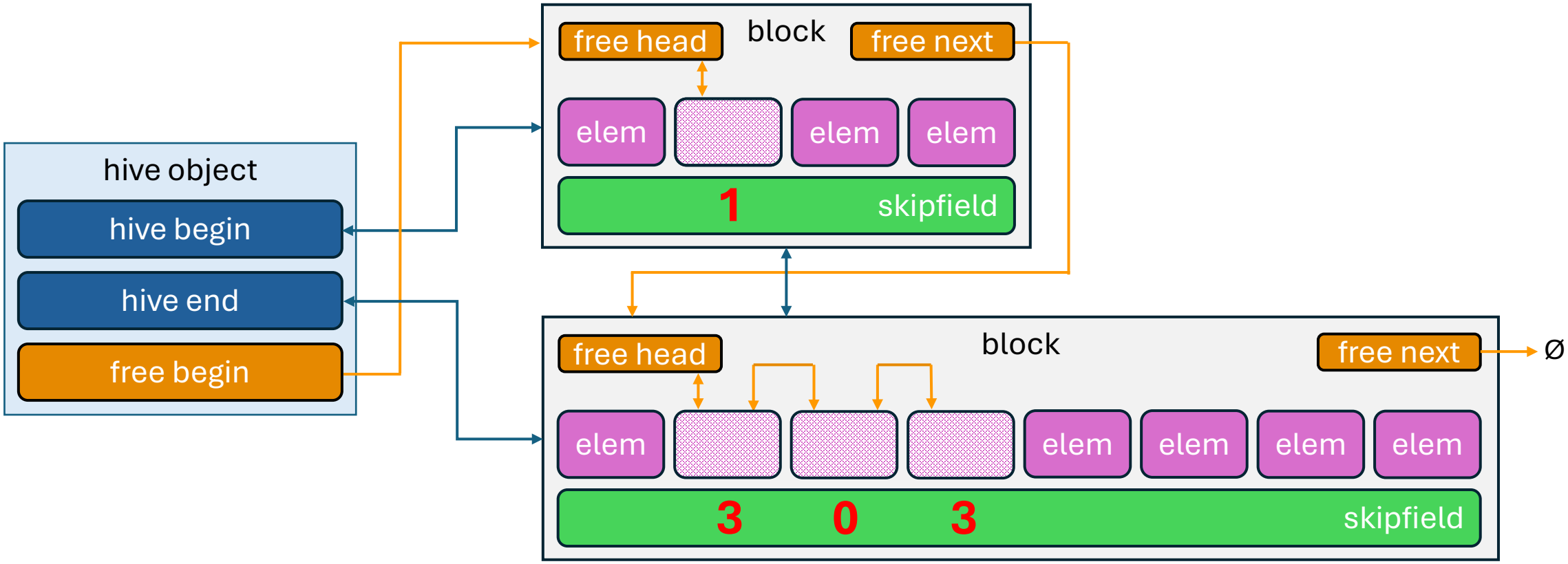
hive – Deleting Elements



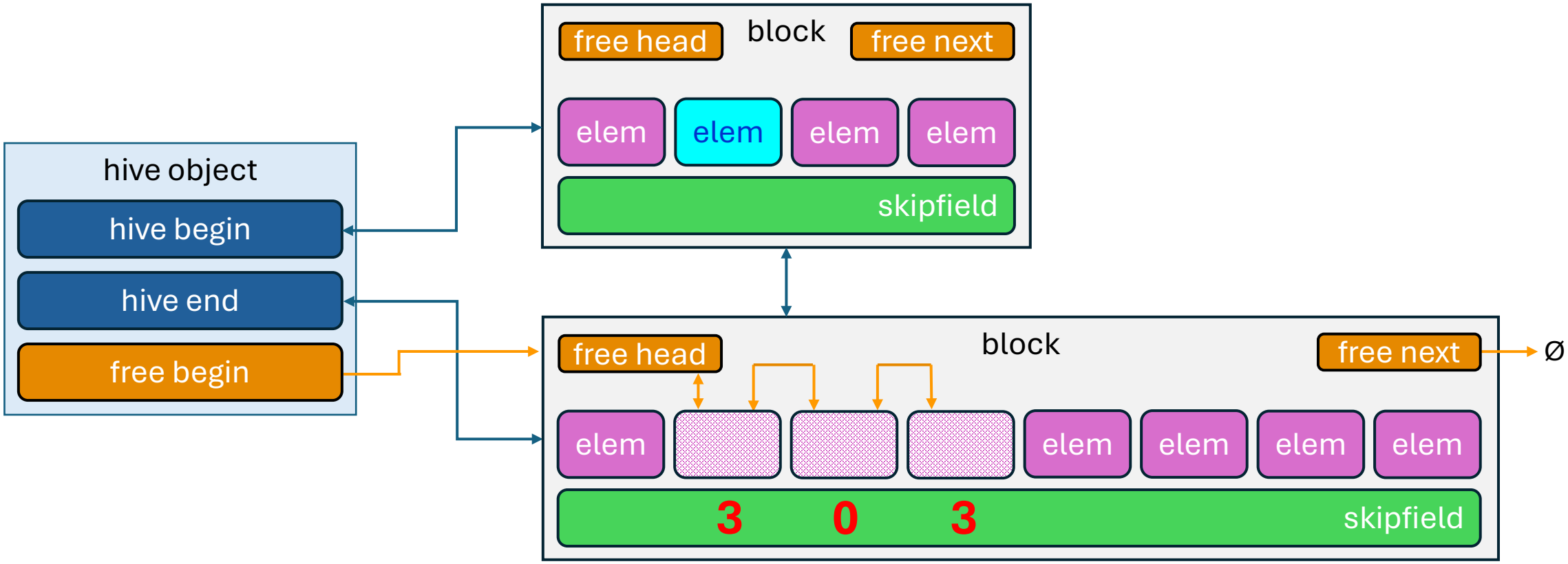
hive – Deleting Elements



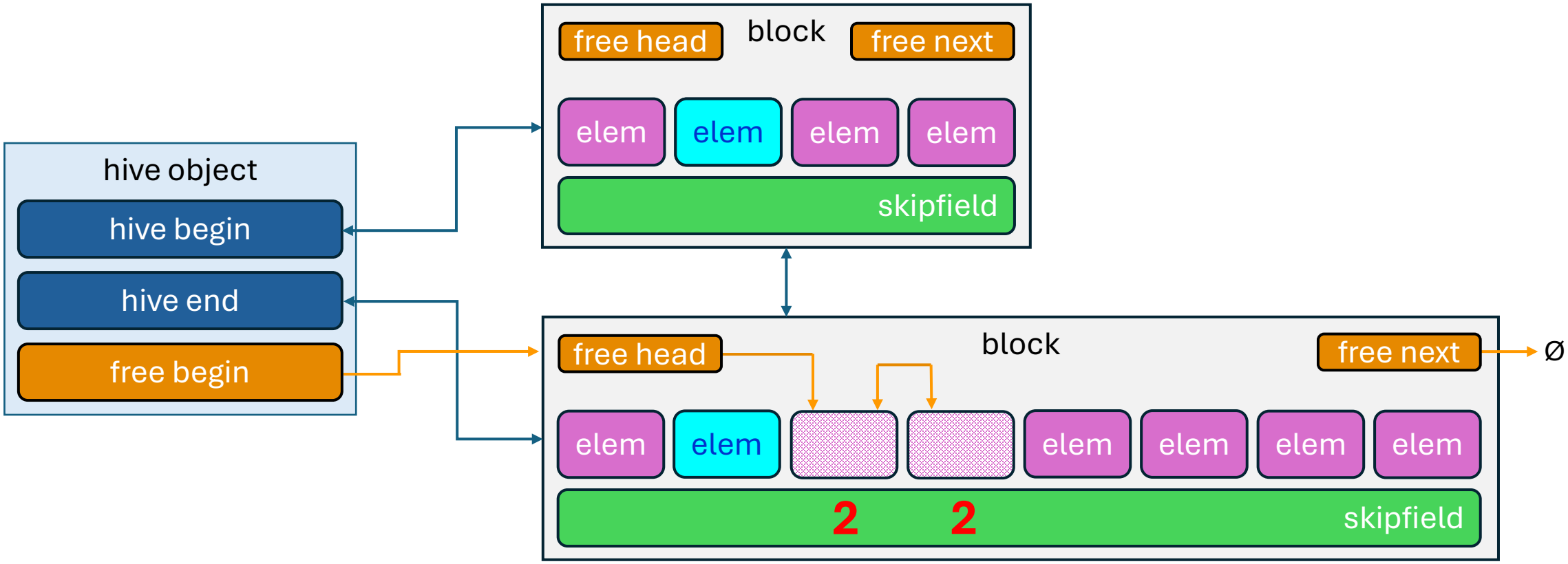
hive – Deleting Elements



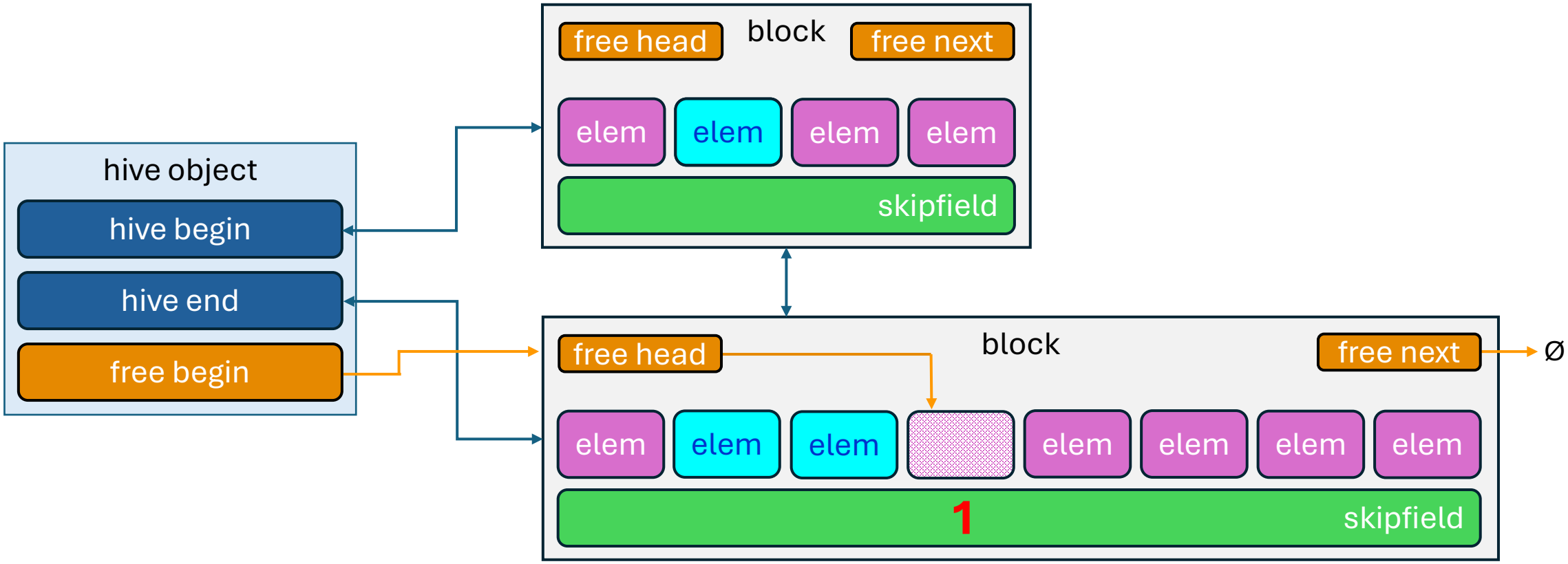
hive – Adding More Elements



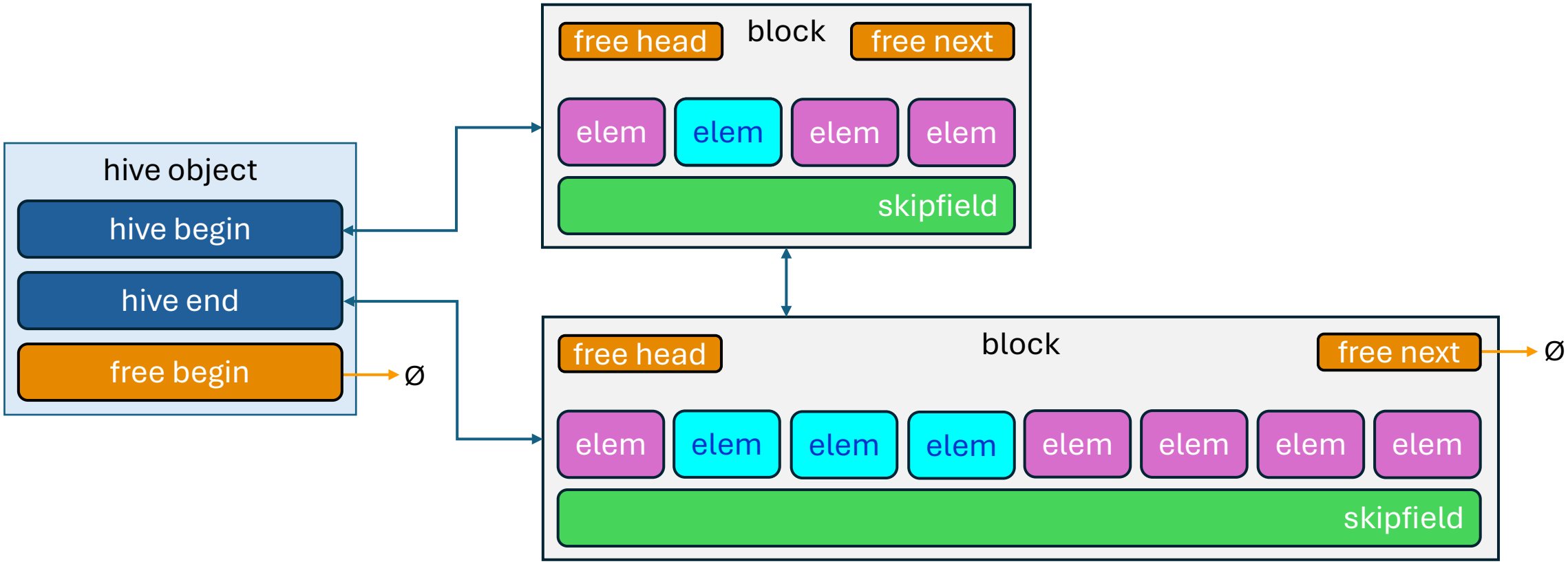
hive – Adding More Elements



hive – Adding More Elements



hive – Adding More Elements



Node-based Containers

- Each element resides in its own separate memory allocation
  - You can think of this as a block-based container with a block size of 1
  - Which provides very smooth linear memory growth
  - But makes for high overhead for small elements
- Locality *and* predictability of reference are terrible
  - Every element is somewhere random in main memory
  - So iteration performance is poor
  - If possible, load node-based containers in one go to improve locality
- But for large elements, these factors are not important
  - The overhead is 2 pointers/element—not a problem for big objects
  - If your elements are big, fetching a page for every element may not be inefficient

Node-based Containers

- Optimal stability
  - No existing elements are ever moved
  - So links are never invalidated
- Objects which cannot be copied *or* moved are well supported
  - Emplace-construct and splice if needed
  - You can’t do this with a **vector** or **deque**
- **list** and **forward_list**
  - The node-based sequence containers
- **map**, **set**, etc…
  - The (ordered) associative containers
- **unordered_map**, **unordered_set**, etc…
  - The unordered associative containers

Node Extraction

- Node-based containers feature fast and stable element manipulation
  - E.g. sorting and tree balancing without moving or copying elements
- **list** has always had **splice**
- Until C++17, sets and maps did not have anything like splicing
- All these operations required creating new elements and deleting old ones
  - Change the key of an element
  - Transfer elements from one container to another
  - Rescue an element if the container was being destructed
  - Move an element out of a set (vs. copying it)
    - Elements in a set are const, and you can’t move from a const object
    - The same applies to the keys of a map

Node Extraction

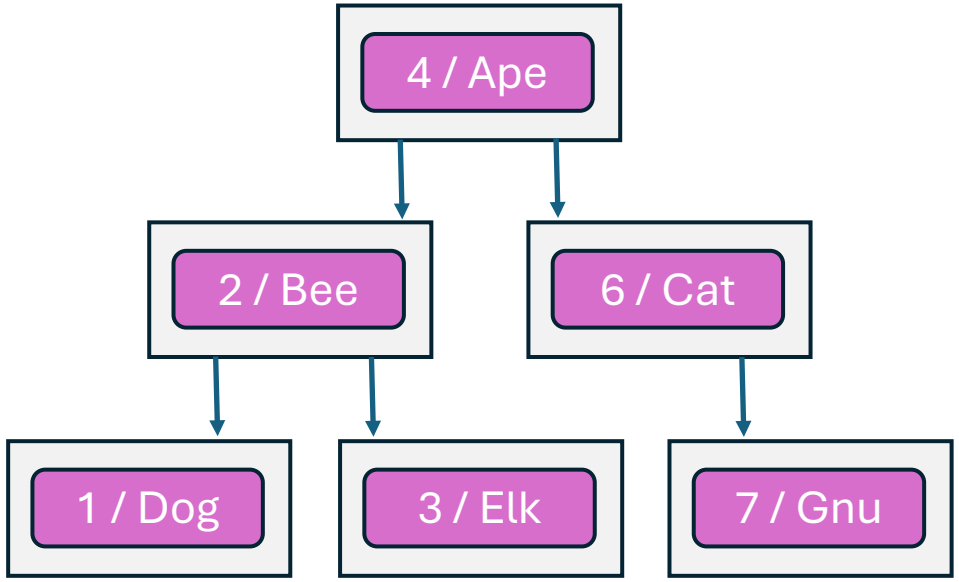
- The C++17 Standard Library has node extraction
- Manipulations of elements can now be done *without* allocation/deallocation
  - And without any (unwanted) moving or copying
- For example:
  - Change the key of an element
  - Transfer an element to another container with compatible nodes
    - Including heterogeneous transfers like **map** to **multimap**
  - Merge one container into another
  - Remove an element and hold it for later use
    - Surviving the destruction of the container
  - Move an element out of a set or map key

Node Extraction

- Node handles provide the mechanism behind all these manipulations
- Node handles may be extracted and inserted
- Node handles allow non-const access to the key of a map or value of a set
- **merge** provides an easy way to combine two compatible containers
  - Including heterogeneous merges like **map** with **multimap**

Changing an Element Key

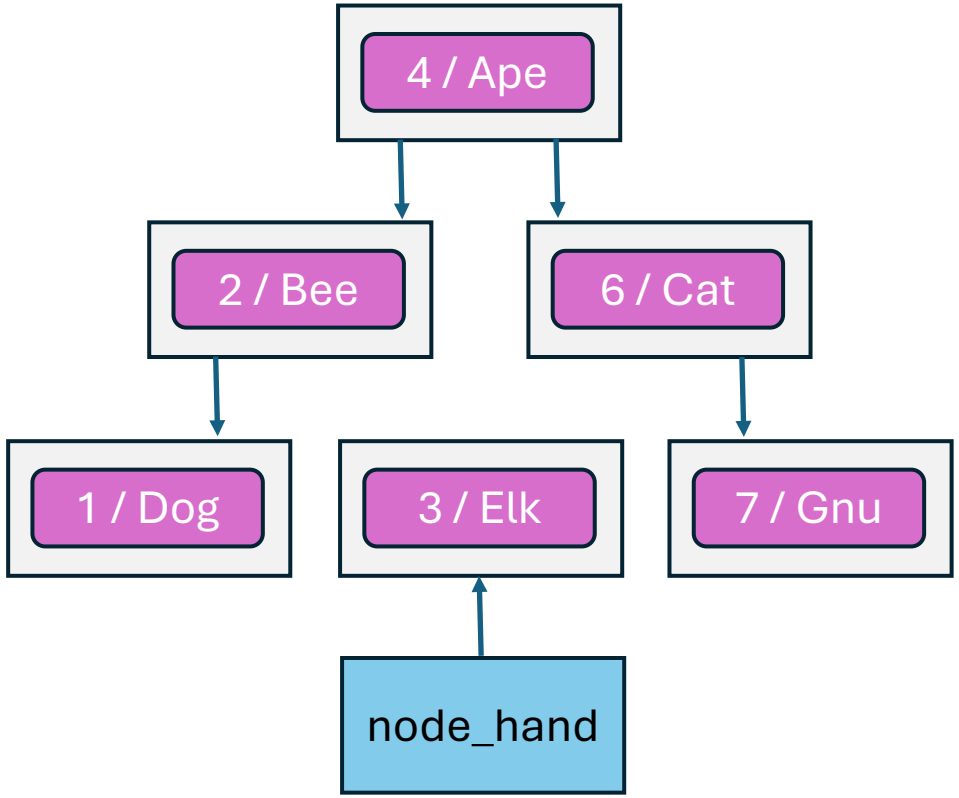
```
map<int, string> m{
   {1,"Dog"}, {2,"Bee"}, {3, "Elk"},
   {4,"Ape"}, {6,"Cat"}, {7, "Gnu"}
};
```



Changing an Element Key

```
map<int, string> m{
   {1,"Dog"}, {2,"Bee"}, {3, "Elk"},
   {4,"Ape"}, {6,"Cat"}, {7, "Gnu"}
};

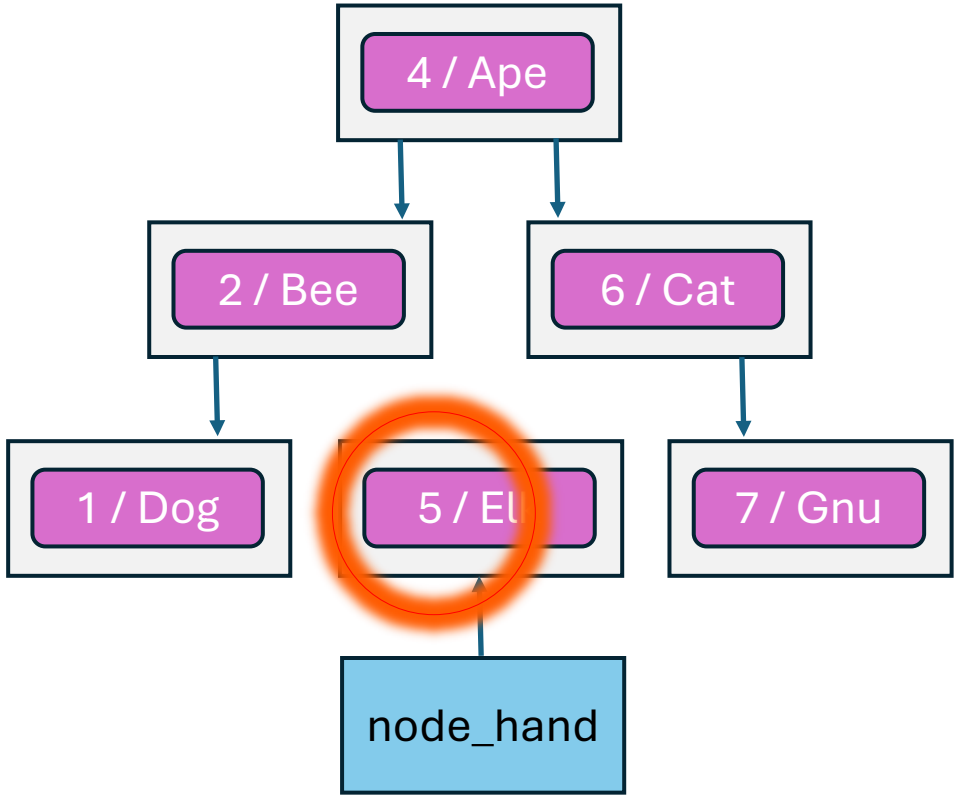
auto node_hand = m.extract(3);
```



Changing an Element Key

```
map<int, string> m{
   {1,"Dog"}, {2,"Bee"}, {3, "Elk"},
   {4,"Ape"}, {6,"Cat"}, {7, "Gnu"}
};

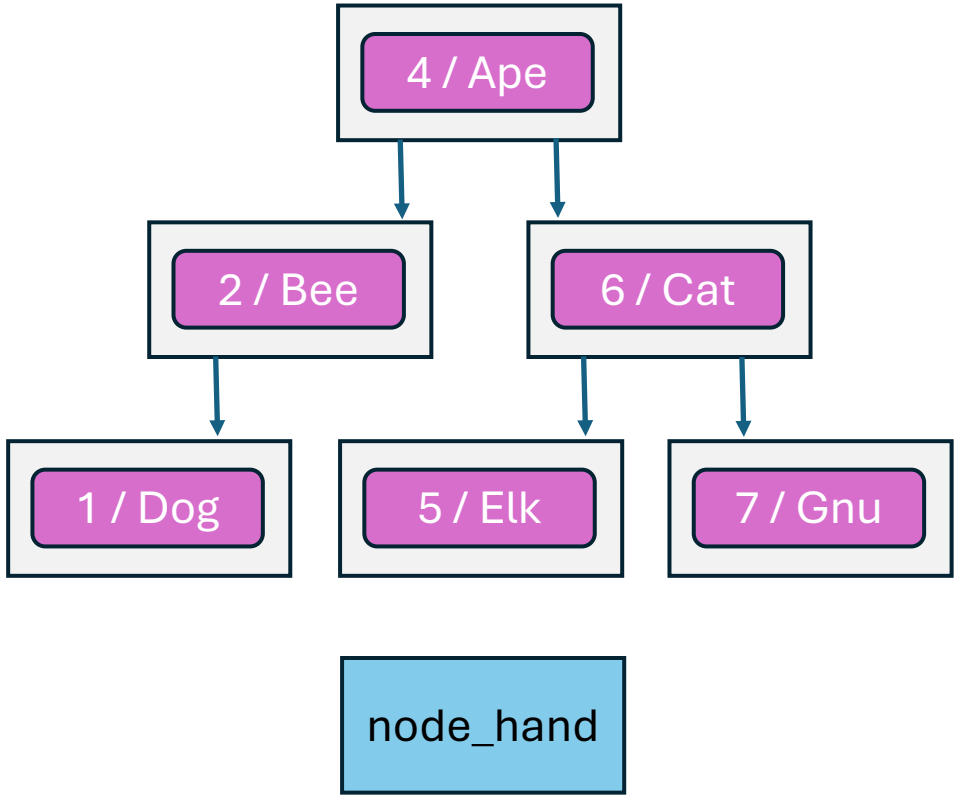
auto node_hand = m.extract(3);
node_hand.key() = 5;
```



Changing an Element Key

```
map<int, string> m{
   {1,"Dog"}, {2,"Bee"}, {3, "Elk"},
   {4,"Ape"}, {6,"Cat"}, {7, "Gnu"}
};

auto node_hand = m.extract(3);
node_hand.key() = 5;
m.insert(move(node_hand));
```



Changing an Element Key

```
map<int, string> m{
   {1,"Dog"}, {2,"Bee"}, {3, "Elk"},
   {4,"Ape"}, {6,"Cat"}, {7, "Gnu"}
};

auto node_hand = m.extract(3);
node_hand.key() = 5;
auto irt = m.insert(move(node_hand));

struct insert_return_type {
    iterator  position;
    bool      inserted;
    node_type node;
};
```

Merging Sets

```
set<int> src{1, 3, 5};
set<int> dst{2, 4, 5};

dst.merge(src);
```

```
src is now: {5}
dst is now: {1, 2, 3, 4, 5}
```

```
set<int> src{1, 3, 5};
multiset<int> dst{2, 4, 5};

dst.merge(src);
```

```
src is now: {}
dst is now: {1, 2, 3, 4, 5, 5}
```

Efficient Factories

```
auto new_record(const char* str)
{
   static int id = 0;
   map<int, string> temp;
   temp.emplace(++id, str);
   return temp.extract(temp.begin());
}

map<int, string> table;
table.insert(new_record("Hello"));
table.insert(new_record("World"));
```

```
table is now:
      {1,"Hello"},
      {2,"World"}
```

Container Adaptors

- Container adaptors are wrappers around containers
  - They have one or more template arguments specifying the underlying container type(s)
  - They provide a particular API and specialized behavior (e.g. a stack interface)
- Adaptors maintain their own container
  - They do not reference your container
  - If you pass your container to the constructor, the adaptor will copy or move it
  - Moving something like a vector into an adaptor gives you control of the initial state
- Adaptors have specific requirements of the container they wrap
  - Various features will not compile if you don’t meet those requirements
  - These requirements limit which containers can be used

Associative Container Adaptors

- Added to the Standard Library in C++23
- Four associative adaptors to match the four associative containers:
  - **flat_set**, **flat_multiset**, **flat_map**, **flat_multimap**
- Implement the same abstractions as their namesakes
  - Using sequences instead of a node-based tree structure
- Wrap sequence containers—one for sets, two for maps
  - Solve the performance problems of using node-based containers
  - Using two separate containers for maps is actually an optimization
- All of them work with **vector**, **deque**, and **inplace_vector**
- All of them default to **vector**
  - As they should

Associative Container Adaptors – Efficient Use

- You can **reserve** and preload **vector**(s), then move them into the flat adaptors

```
vector<int> k;
k.reserve(256);
k = {1, 2, 3, 4};

flat_set myset(move(k));
```

```
vector<int> k;
k.reserve(16);
k = {1, 2, 3, 4};

vector<string> v;
v.reserve(16);
v = {"A", "B", "C", "D"};

flat_map mymap(move(k), move(v));
```

Associative Container Adaptors – Efficient Use

- You can call **reserve** or **shrink_to_fit** on containers in the adaptor

```
flat_set myset = ...

auto c = move(my_set).extract();

c.reserve(512);

my_set.replace(move(c));
```

```
flat_map mymap = ...

auto c = move(mymap).extract();

c.keys.shrink_to_fit();
c.values.shrink_to_fit();

mymap.replace(move(c.keys),
              move(c.values));
```

Example #3 – Problem and Solution

- I needed to extract digital road map products from a complex GIS database
  - There were a lot of relationships between the tables
  - Complicated joins in the database ran very slowly
  - But reading the tables into local memory was very fast
- I represented each table in memory with a **map**
  - The key was the table primary key
  - The value was a class holding all the row information
  - All the relationships became links (pointers)
- This worked pretty well!
  - My code ran almost 100x faster than the code it was replacing
    - From 3 *weeks* (continuous runtime) to overnight
  - This required filling memory completely
    - Which worked fine because **map** is node-based

Example #3 – But Next Time…

- This was a controlled environment
  - The tables were static—this was a read-only operation
  - The size of the tables was available in advance from the database
- Was I leaving both time and space performance on the table?
- What if I used sorted vectors?
  - A **vector** could be pre-reserved to the exact size for each table
    - Thus eliminating the filling memory problem
  - Each **vector** could be loaded, then sorted
    - Or insertion sorted if the code would otherwise be waiting for the database
  - Elements are stable once you stop modifying the **vector**
    - Links to the elements could be created once the tables were stable
  - The data would take up a lot less space (which would make it faster)
  - Iteration of tables would certainly be faster
  - Pointer references might be faster due to adjacency of table elements

Example #3 – And Now We Can Run Flat

- With C++23 I could use a **flat_map**
  - Based on either a pre-reserved **vector** or **deque**
  - Or even a **vector** for the keys (IDs) and a **deque** for the values (row records)
- This is a clean, drop-in solution
- **flat_map** is awesome!

sequence

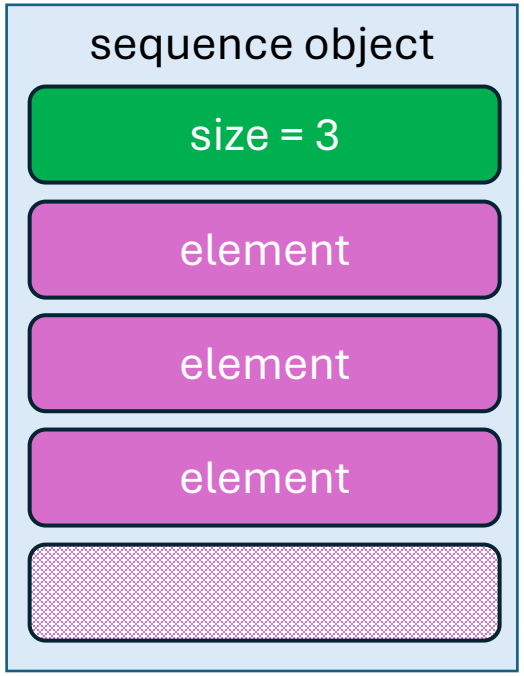
- **sequence** is a proposed future contiguous sequence container
  - See the links at the end for references
- Disclaimer:
  - The proposal is mine
- It looks a lot like **vector** but provides compile-time user control of:
  - Memory handling
  - Element positioning
  - Growth behavior
- It does *not* have a **vector<bool>** specialization
  - **sequence<bool>** gives you a sequence of **bool**, as requested
  - As I said in Part 1, use a proper bitset

sequence – Memory Handling

- There are four storage modes:
  - LOCAL – Elements reside in the container object, the capacity is fixed at compile time
    - Like **inplace_vector** or **boost::static_vector**
  - FIXED – Dynamically allocated block, but the capacity is fixed at compile time
    - It can't grow (or shrink), unlike **vector**
  - VARIABLE – Dynamically allocated block that can grow in size
    - Like **vector**
  - BUFFERED – Dynamically allocated block with user-controlled small buffer optimization
    - Like **clump** or **boost::small_vector**
- Cross-mode operations are supported
  - And will be as efficient as possible
  - For example: blocks can be moved between VARIABLE and BUFFERED modes

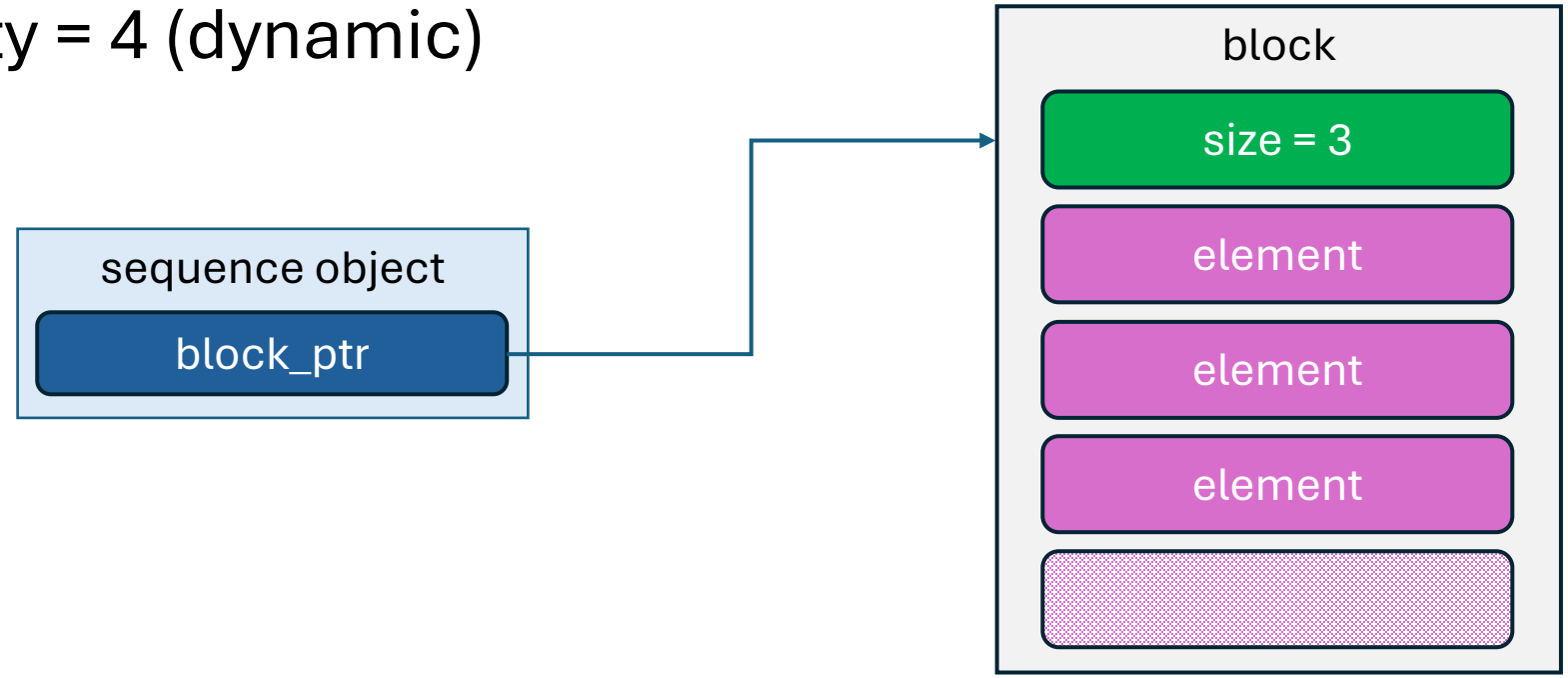
sequence – LOCAL Storage

fixed capacity = 4 (local)
size = 3



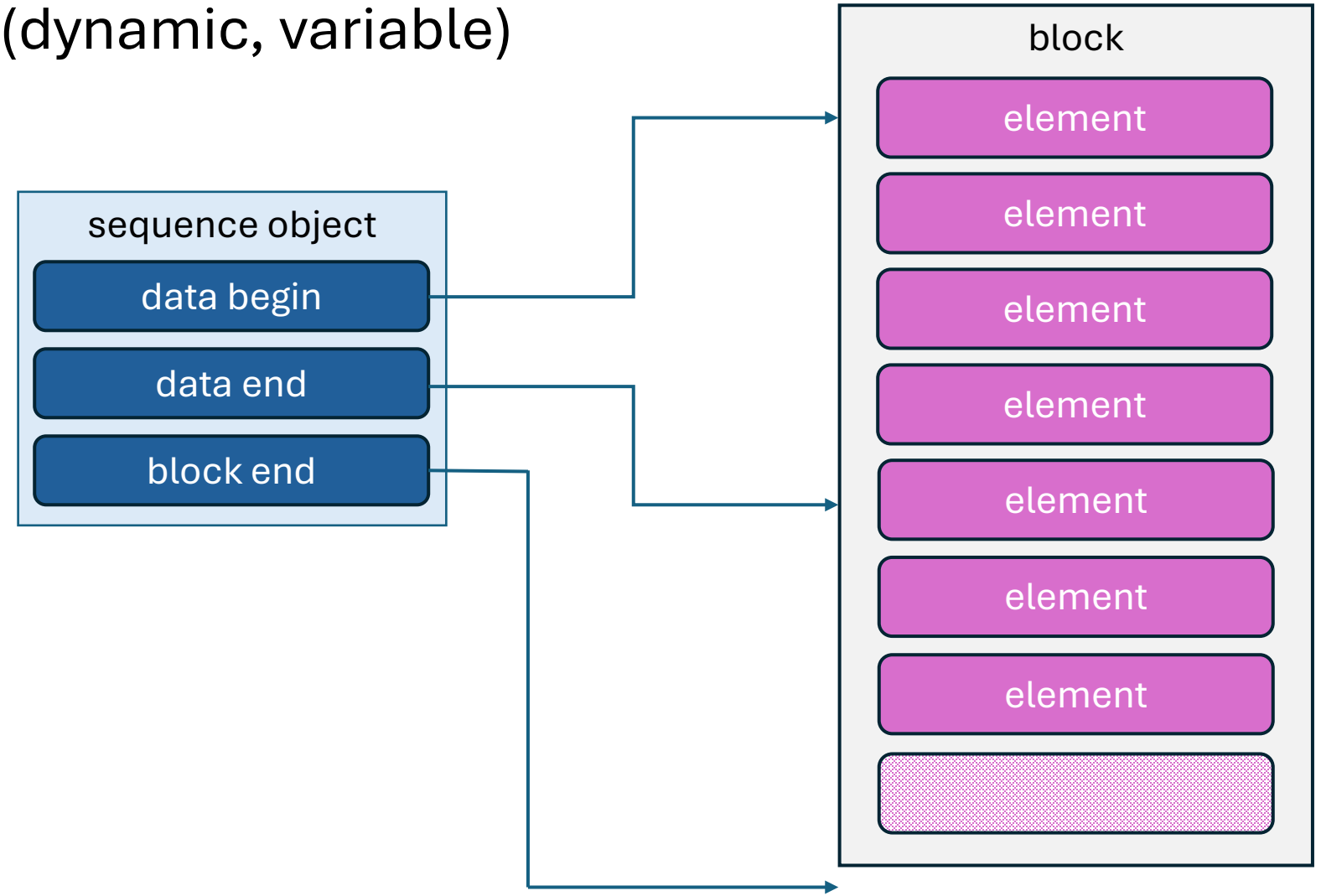
sequence – FIXED Storage

fixed capacity = 4 (dynamic)
size = 3



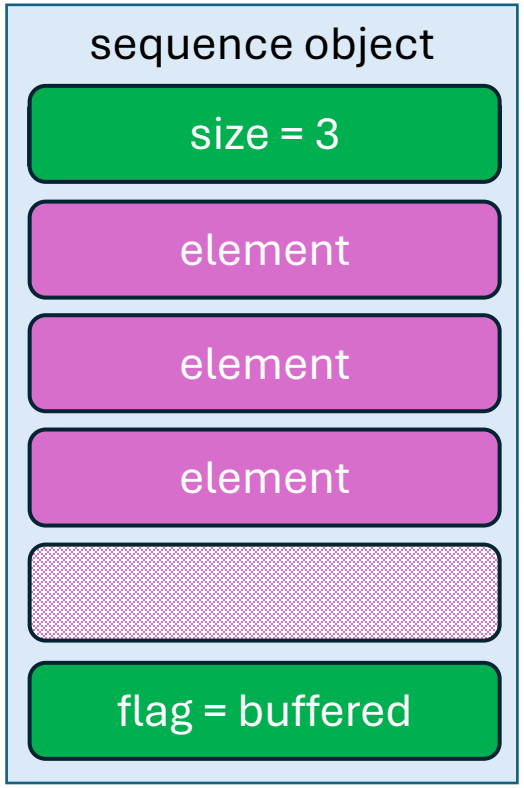
sequence – VARIABLE Storage

capacity = 8 (dynamic, variable)
size = 7



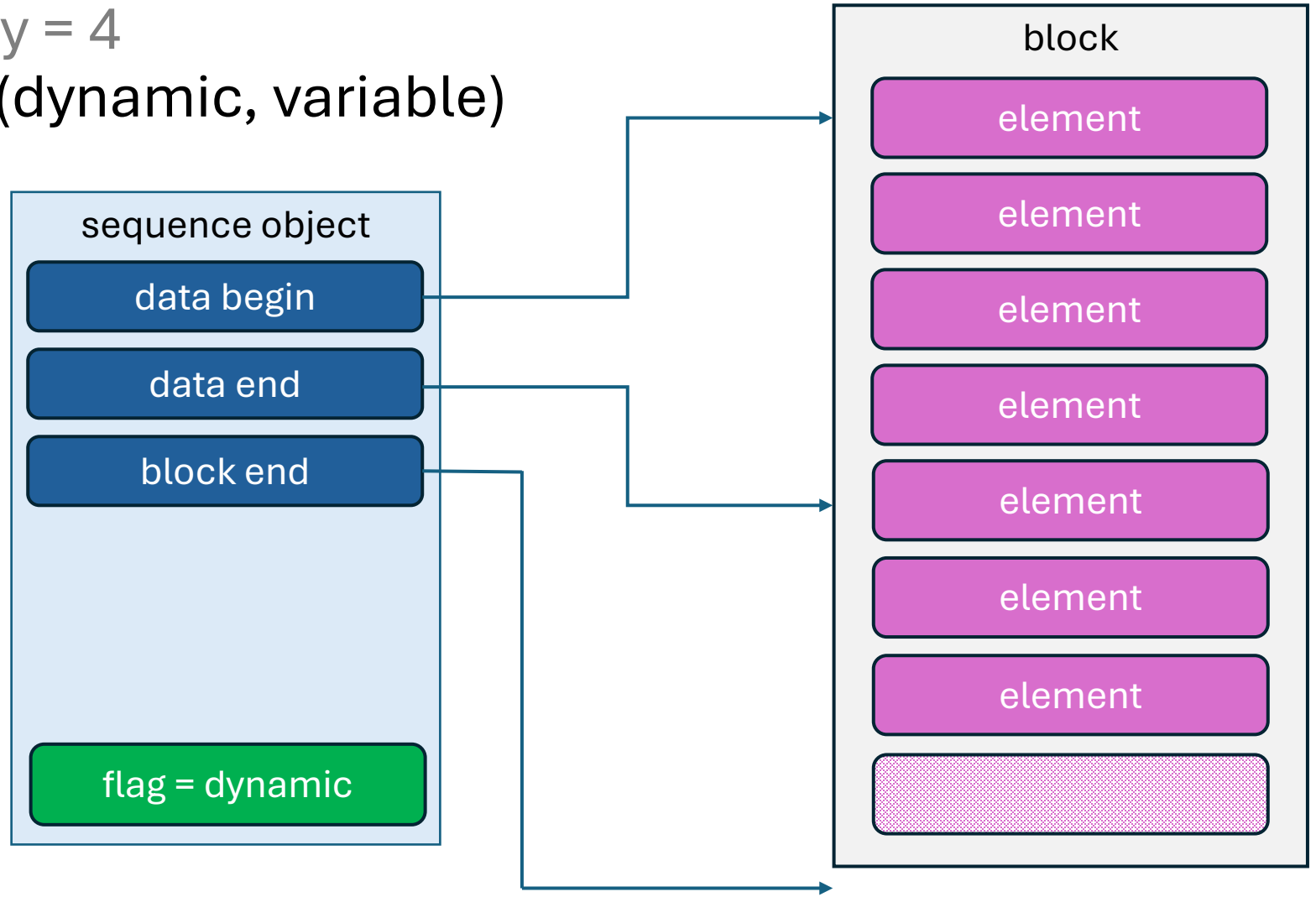
sequence – BUFFERED Storage

fixed capacity = 4 (local)
size = 3



sequence – BUFFERED Storage

fixed capacity = 4
capacity = 8 (dynamic, variable)
size = 7



sequence – Memory Handling

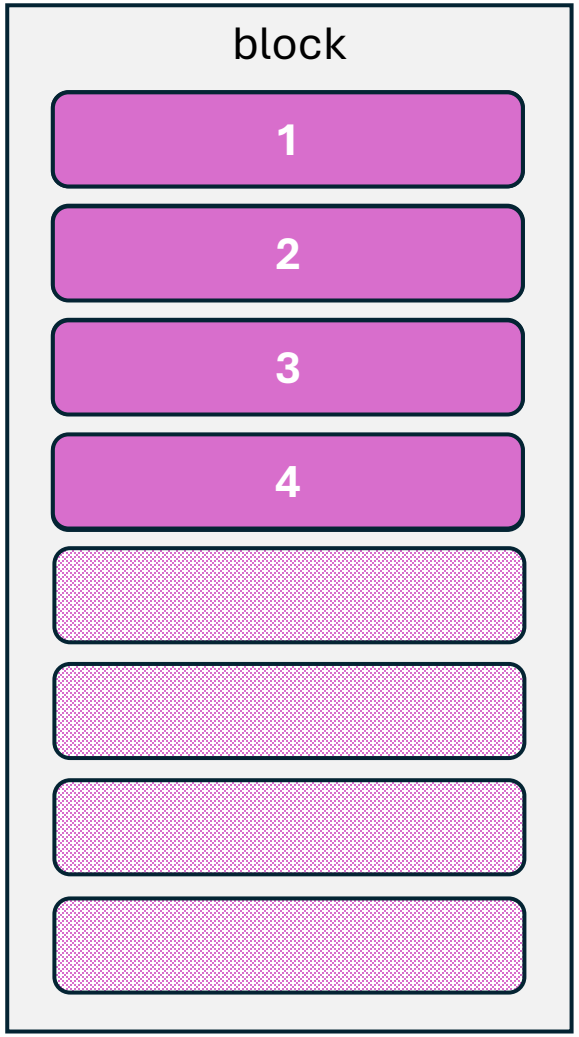
- The fixed capacity value is used as the initial capacity in VARIABLE mode
  - This eliminates the need for the “construct and immediately reserve” pattern
  - And it’s better because it’s lazy—the allocation does not happen until the first insertion
  - So empty sequences remain blockless until and unless a block is needed
- The size field type is user-controlled
  - All the storage modes except VARIABLE use a size field
    - Or two in the case of MIDDLE element positioning
  - It can be a valuable space optimization to be able to control the type of this field
    - Consider a local sequence of 15 chars: you probably don’t want an 8 byte size field
- It is permitted to call **reserve** in any mode
  - So long as the argument is not greater than the fixed capacity for LOCAL and FIXED
  - This will cause FIXED to allocate a block of the fixed capacity size
    - The provided argument is ignored
  - This is to allow changing container modes without changing client code

sequence – Element Positioning

- There are three element positioning modes:
  - FRONT – Free space in the block comes *after* the elements
    - Like **vector**
  - BACK – Free space in the block comes *before* the elements
  - MIDDLE – Free space in the block may be found *before and/or after* the elements
    - The first element is placed in the middle of the block
    - The elements are shifted toward the middle if necessary
    - The block is filled before a new block is allocated
    - Optimized for a double-ended queue
- Front and back operations are available in all modes
  - Unlike **vector**, which does not provide front operations

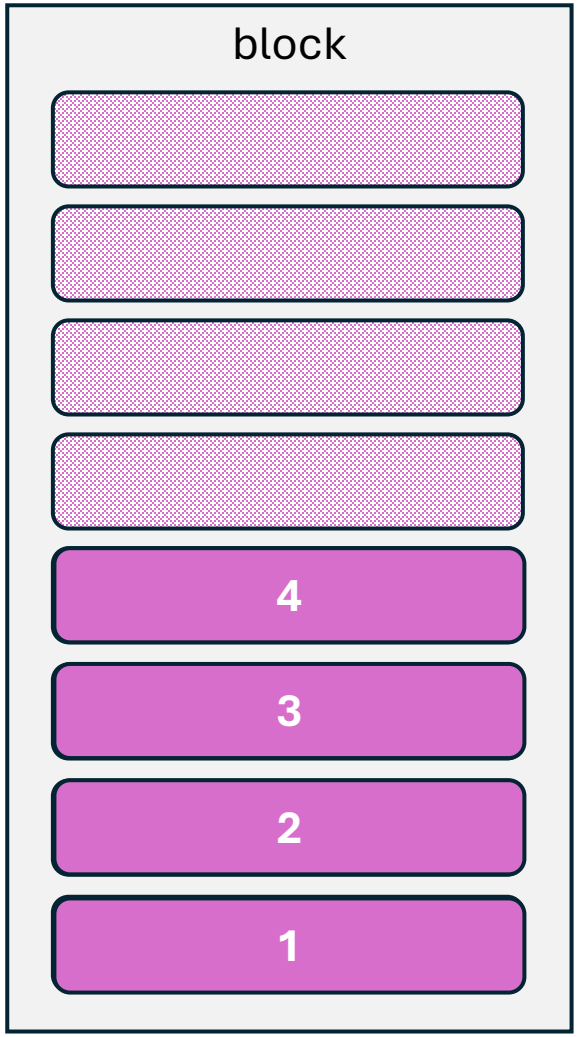
sequence – FRONT Location

push_back

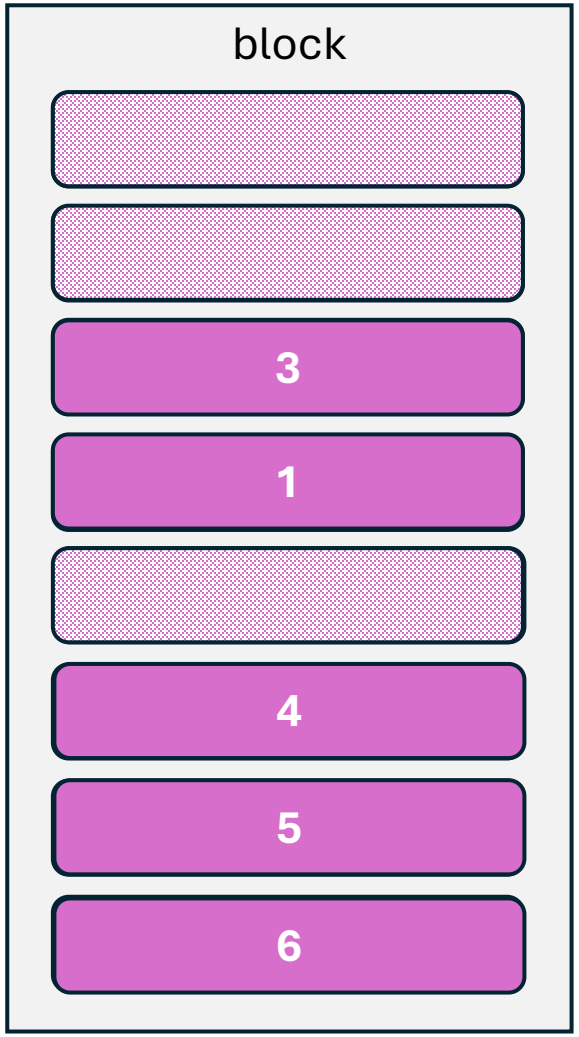


sequence – BACK Location

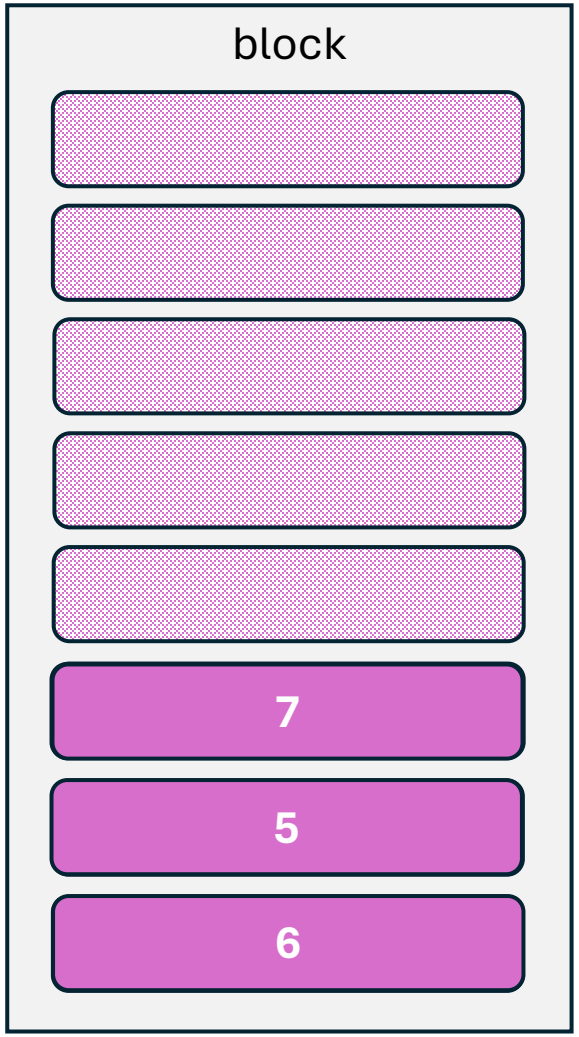
push_front



sequence – MIDDLE Location



sequence – MIDDLE Location



sequence – Growth Behavior

- There are three capacity growth modes:
  - LINEAR – Capacity grows by a fixed, user-defined number of elements
  - GEOMETRIC – Capacity grows by a user-defined factor
    - Like **vector**, but you get to choose the factor
  - VECTOR – This provides a hook so library implementors can exactly replicate **vector**
    - Even if GEOMETRIC mode doesn't exactly model their **vector**'s growth

sequence – Member Functions

- **free** – member function which erases all elements *and* frees the block
  - With **vector**, you have to call **clear** then **shrink_to_fit** to get this result
- **is_dynamic** – member function which returns:
  - True if the elements are dynamically allocated on the heap
  - False if they are buffered in the container object
  - Does not (necessarily) return false if the container is in a default constructed state
    - To determine this, use: `capacity() == 0`

sequence – Motivation

- Variety – There are several features not otherwise available in the SL
- Flexibility – You can match the behavior to the problem you are trying to solve
- Tuning – It is easy to tune for performance
  - The API is consistent regardless of user settings
  - Your container can be switched between various modes
  - Specific behaviors can be adjusted
  - You can find the best results with no client code changes (or compromises)
- Improvements – There are several problems with **vector** and **inplace_vector**
  - These issues can’t (or won’t) be changed in existing containers
  - But they can be reconsidered in a new container
  - E.g. front operations

sequence – Motivation

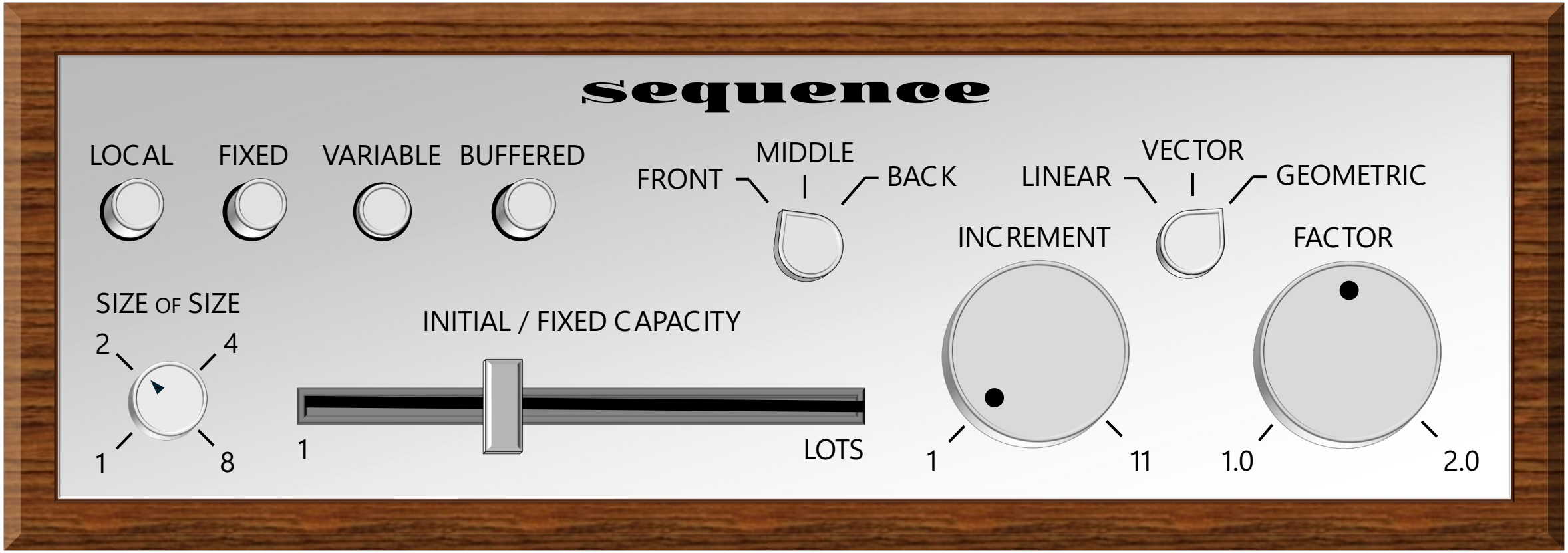
- Simplification – It subsumes several existing and proposed containers
  - **vector**
    - **vector** can be defined in terms of **sequence** (VARIABLE storage)
  - **inplace_vector**
    - **inplace_vector** can be defined in terms of **sequence** (LOCAL storage)
    - Unfortunate timing: we didn't have this so we needed **inplace_vector**
  - **clump**
    - A proposed future contiguous sequence container
    - Combines the properties of **vector** and **inplace_vector** (like **boost::small_vector**)
    - Implements what is often called the small buffer optimization (SBO)
    - **clump** can be implemented with **sequence** (BUFFERED storage)
    - Not a timing problem yet because the **clump** proposal is not currently active
    - See the link at the end for Nevin Liber's proposal

sequence – Specific Use Cases

- MIDDLE mode will be great for queues
  - And the **queue** adaptor works fine with **sequence**
- There is an interesting irony about **deque**
  - A popular design of the block table in **std::deque** is a double-ended queue maintained in a contiguous block
  - That is exactly what **sequence** in MIDDLE-mode provides
- The new **flat_map** (etc.) adaptors will benefit from **sequence**
- There are many, many uses for a local vector
  - So many that we now have **inplace_vector**
  - This is, of course, an argument both for and against standardizing **sequence**
- There are many, many uses for a vector with SBO
  - So many that we have both a proposal and a boost library

sequence – Objections

- We already have **vector** and **inplace_vector**
  - True, and if we get **clump** we'll have yet another single-purpose tool
  - The idea behind **sequence** is to have a multi-purpose tool
- There are too many containers already
  - I completely understand concerns over the cost of another container
  - This proposal is meant to replace an existing proposal for another container (**clump**)
  - And offer more facilities which might be requested in yet another container proposal
  - While it increases the API surface area, it reduces the user's cognitive load
- Controlling behavior with template arguments is not a good library design
  - I don't understand this argument, so I'd welcome explanations
- This is all just too complicated for regular programmers
  - Naw, I think C++ programmers are pretty smart
  - And if you can operate a 1970's era stereo, you can run this thing


sequence
LOCAL
FIXED
VARIABLE
BUFFERED
FRONT
MIDDLE
BACK
LINEAR
VECTOR
GEOMETRIC
INCREMENT
FACTOR
SIZE OF SIZE
2
4
1
8
INITIAL / FIXED CAPACITY
1
LOTS
1
11
1.0
2.0

Links

- Related talks
  - Timur Doumler: *Want fast C++? Know your hardware!* (CppCon 2016)
    www.youtube.com/watch?v=BP6NxVxDQIs
  - Fedor Pikus: *Unlocking Modern CPU Power - Next-Gen C++ Optimization Techniques* (C++Now 2024)
    www.youtube.com/watch?v=wGSSUSeaLgA
  - Mathieu Ropert: *Heaps Don't Lie, Guidelines for Memory Allocation in C++* (ACCU 2025)
    www.youtube.com/watch?v=74WOvgGsyxs
  - Jon Kalb: *Alex Stepanov, Generic Programming, and the STL* (ACCU 2025)
    www.youtube.com/watch?v=d_DlHYX5w5w
  - Alan Talbot: *Moving Faster: Everyday efficiency in modern C++* (CppCon 2018)
    www.youtube.com/watch?v=EovBkh9wDnM
  - Alan Talbot: *How to Choose the Right Standard Library Container* (CppCon 2019)
    www.youtube.com/watch?v=yjPKVOYcw28
  - Alan Talbot: *New (& Old) C++ Standard Library Containers - How to Choose the Right Container in C++26 and Beyond* (ACCU 2025)
    www.youtube.com/watch?v=TtbYGico7bI

Links

- **hive**
  - Matthew Bentley: *Colonies, performance and why you should care* (CppCon 2016) www.youtube.com/watch?v=wBER1R8YyGY
  - Matthew Bentley: *How to: Colony* (CppCon 2021) www.youtube.com/watch?v=V6ZVUBhls38
  - Matthew Bentley: *Introduction of std::hive to the standard library* www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p0447r28.html
  - Matthew Bentley: github/plf_hive github.com/mattreecebentley/plf_hive
  - Matthew Bentley: PLF C++ Library - plf::colony plflib.org/colony.htm
- **flat_map**
  - Zach Laine: *A Standard flat_map* www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0429r3.pdf

Links

- **clump**
  - Nevin Liber: *Clump – A Vector-like Contiguous Sequence Container with Embedded Storage*
    www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0274r0.pdf
- **sequence**
  - sequence – A contiguous sequence container with adjustable performance characteristics
    www.github.com/talbotcpp/sequence
  - Alan Talbot: *A Direction for Vector*
    www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3147r1.pdf
  - Alan Talbot: *Vector Front Operations*
    www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0563r0.pdf